Your six-phase assignment guide

Worked example: steel plate fault classification

A complete learning reference covering dataset preparation, five models, six metrics, a Streamlit evaluation app, repository organization, and the final submission process.

Reference project	Verified facts
Dataset	UCI Steel Plates Faults: 1,941 rows, 27 inputs, 7 classes
Evaluation	1,552 training rows; 389 held-out test rows; seed 42
Preferred baseline	SVM by training-only cross-validation macro F1
Reference test result	SVM accuracy 0.7635; macro F1 0.7810; AUC 0.9566
Execution context	ChatGPT execution environment, not BITS Virtual Lab

Use this as learning support

The assignment explicitly permits AI for learning support, not direct copy-paste submissions. Read the explanations and code, complete the notebook checkpoints, make and justify your own implementation choices, and write your own analysis after running the work in BITS Virtual Lab.

The reference project has been trained and checked locally. It does not include a real repository link, a live Streamlit Community Cloud deployment, or a BITS Lab screenshot. Those must come from your own work and accounts.

Start here

1. Extract the project ZIP. Open the notebook inside steel_faults_learning, next to ml_core.py, train.py, and app.py.

2. Follow Steps 1-6 in order. Use the runnable reference to check your understanding; complete the reflection prompts and your own experiment.

3. Build the final PDF only after completion. It must contain real links and real lab evidence, followed by the full final README.

Assignment deadline stated in the uploaded PDF: 8 November 2026, 23:59. The PDF does not specify a timezone. It states that drafts are not accepted.

STEP 1

Choose and understand the data

Steel Plates Faults is a public UCI multiclass dataset of numeric measurements describing steel defects. It is relevant to industrial inspection and exceeds both dataset-size requirements. The task uses measured features, not raw images.

Requirement	Minimum	This dataset
Input features, excluding targets	12	27
Instances	500	1,941
Classification task	Binary or multiclass	Seven mutually exclusive fault classes

Avoid target leakage during conversion

The original source has 27 predictors and seven class indicator columns. prepare_data.py verifies that exactly one indicator is active in each row, converts those indicators into one target label, and removes every indicator from the predictors. Including an original indicator in X would expose the answer.

```
target = raw[seven_target_columns].idxmax(axis=1)
X = raw[FEATURE_COLUMNS]  # only the 27 input features
```



The reference inspection found no missing cells and no duplicate feature rows. The smallest class, Dirtiness, has 55 examples; Other_Faults has 673. Macro metrics therefore complement overall accuracy.

Checkpoint: explain the target conversion, the meaning of the 27-feature count, and why class imbalance affects your choice of metric.

Dataset: UCI Steel Plates Faults. Buscema, M., Terzi, S., & Tastle, W. (2010). DOI: 10.24432/C5J88N. License: CC BY 4.0. Original label spelling K_Scatch is retained.

Train five comparable baselines

1. Split first. Create one stratified 80/20 split. Every classifier receives the same training and test rows. Keep test_data.csv in raw feature units and include its target column.

2. Fit preprocessing on training rows. Use StandardScaler inside a Pipeline for LR, KNN, SVM, and Gaussian NB. The tree uses unscaled inputs. Pipelines prevent full-dataset scaling from leaking test information.

3. Select with training-only CV. Use five stratified folds within the 1,552 training rows. Pick the preferred fixed baseline by mean macro F1. The reference does not tune parameters using held-out scores.

4. Fit, evaluate, and save. Fit each model on all training rows, evaluate on the same 389 held-out rows, and save the entire pipeline in model/.

Classifier	Fixed reference settings
Logistic Regression	C=1.0; max_iter=5000
Decision Tree	max_depth=10; min_samples_leaf=3
KNN	n_neighbors=7; weights=distance
Gaussian Naive Bayes	var_smoothing=1e-9
Support Vector Machine	RBF kernel; C=10; gamma=scale; probability=True

State the metric convention in every output

Metric	Definition used
Accuracy	Correct predictions divided by total test examples.
AUC	Macro one-versus-rest ROC-AUC from all class probabilities.
Precision	Mean of the seven per-class TP / (TP + FP) values.
Recall	Mean of the seven per-class TP / (TP + FN) values.
F1	Mean of the seven per-class F1 values.
MCC	Generalized multiclass Matthews correlation coefficient.

For AUC, probability-column order must match model.classes_. A hard predicted class is not a probability score. Macro F1 is not the harmonic mean of macro precision and macro recall. Undefined per-class P/R/F1 values use zero_division=0.

If a test upload lacks any class, the chosen all-seven-class AUC is undefined; the app displays N/A with an explanation. The complete reference test set contains all classes.

STEP 2 - RESULTS

Read the reference evidence

These are actual results from the included implementation, executed outside BITS Lab. Recompute and discuss your own results after adapting the project and running it in the required environment.

ML model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.7275	0.9379	0.7613	0.7305	0.7418	0.6487
Decision Tree	0.7481	0.9056	0.7784	0.7364	0.7551	0.6728
KNN	0.7558	0.9251	0.7453	0.7687	0.7546	0.6884
Naive Bayes	0.6093	0.9225	0.6333	0.7169	0.6339	0.5484
SVM	0.7635	0.9566	0.8018	0.7674	0.7810	0.6949

Precision, recall and F1 are macro averages. AUC is macro one-versus-rest. All values use the same 389 held-out rows.

Model	Training CV macro F1 mean	Fold standard deviation
Logistic Regression	0.7220	0.0263
Decision Tree	0.7379	0.0132
KNN	0.7676	0.0222
Naive Bayes	0.6153	0.0264
Support Vector Machine	0.7760	0.0187

What the numbers support

Logistic Regression: macro F1 is 0.7418. Its largest confusion is Bumps becoming Other_Faults: 30 examples.

Decision Tree: macro F1 is 0.7551. It finds 74/78 K_Scatch examples but only 12/32 Pastry examples.

KNN: macro F1 is 0.7546 and macro recall is 0.7687, slightly above SVM. Its 1.0 training accuracy is affected by self-neighbors in distance-weighted resubstitution.

Gaussian NB: macro F1 is 0.6339. Only 34/135 Other_Faults examples are identified correctly. Dependence among features is a plausible explanation to investigate, not an established cause.

SVM: highest CV mean macro F1 and highest reference test macro F1. Test accuracy is 297/389 = 0.7635. It still confuses Bumps and Other_Faults.

Conclusion for this experiment: SVM is the preferred baseline under the declared CV macro-F1 rule. The close SVM/KNN CV scores do not establish statistical superiority. No result here proves performance on a different plant or camera.

STEPS 3 AND 4

Organize the repository and environment

Required path	Purpose
app.py	Interactive Streamlit test-data evaluation.
requirements.txt	Pinned dependencies for the tested Python 3.12 reference.
README.md	Required sections a-d, metrics, observations, genuine links.
test_data.csv	27 feature columns and the true target; held-out rows only.
model/	All five .joblib pipelines plus metadata.json.
train.py and ml_core.py	Training implementation, preprocessing, and shared metrics.
prepare_data.py and data/	UCI conversion and reproducible dataset provenance.
Notebook and results/	Explained experiment and exported comparison/reports.

Run from the extracted project directory in BITS Lab

```
python --version
python -m pip install -r requirements.txt
python train.py --execution-context "BITS Virtual Lab"
python -m streamlit run app.py
```

Use the BITS label only when the command actually runs inside the lab. A metadata label is not evidence. Capture the required genuine screenshot separately. The notebook also needs requirements-notebook.txt.

Package	Tested version	Package	Tested version
streamlit	1.52.0	scikit-learn	1.8.0
numpy	2.3.5	pandas	2.2.3
scipy	1.17.0	joblib	1.5.3
matplotlib	3.10.8	Python	3.12

Create your actual GitHub record

Create a repository in your own account, commit your real work as you complete it, and upload the required files. Preserve model/ and test_data.csv. The supplied .gitignore excludes virtual environments, caches, and Streamlit secrets.

Use compatible training and deployment versions. If you must change the ML stack for the lab, retrain the pipelines and update the dependency file together. Do not ignore saved-model version mismatches.

STEP 5

Write the README in the required order

Heading	What your final README must contain
a. Problem statement	Your objective, prediction target, and motivation.
b. Dataset description	Public source, 1,941 rows, 27 inputs, seven classes, class counts, preprocessing, and evaluation split.
c. GitHub Repository Link	Your real working repository link. Add the real Streamlit app link for convenient evaluation.
d. Models used	Five classifiers, six-metric table, metric definitions, model settings, observations for each model, and an overall winner.

A worked class-wise calculation

For reference SVM and the Dirtiness class, the confusion matrix gives TP=8, FP=0, and FN=3. There are 11 true Dirtiness examples.

```
Precision = 8 / (8 + 0) = 1.0000
Recall    = 8 / (8 + 3) = 0.7273
F1        = 16 / (16 + 0 + 3) = 0.8421
```

The perfect precision says every predicted Dirtiness example was correct. It does not mean all Dirtiness examples were detected: three were missed. This is a stronger observation than simply calling the model accurate.

Write an observation using evidence and a limitation

Use a pattern such as: identify a score, point to a class-level strength or failure, explain a plausible reason as a hypothesis, and state one useful next experiment. Keep the training-only selection rule separate from the descriptive held-out comparison.

1. For each classifier, state macro F1 and at least one useful class-level result from your run.

2. For the overall winner, name your selection metric and connect the choice to the CV results and intended use.

3. For your own contribution, document a justified experiment or error analysis, its outcome, and what you learned. Changing variable names or seeds does not substitute for understanding.

4. For the submitted PDF, include the full revised README text and its tables. A README link alone does not satisfy this requirement.

The supplied README has reference results and explicitly pending external links. Replace pending fields and develop your own discussion before creating the final submission.

STEP 6

Deploy and verify the Streamlit app

The app reads the selected test CSV, validates its schema, loads saved pipelines, predicts, and computes metrics. No model or scaler is fitted on uploaded data.

Assignment feature	Reference implementation
CSV upload	27 required features plus the categorical target.
Model dropdown	All five required classifiers.
Evaluation metrics	Accuracy, AUC, macro P/R/F1, and multiclass MCC.
Confusion matrix or report	Both are provided, with class supports.
Different models visible	All-five comparison on the same current test rows.

Verify the local/lab application

Run the app, select all five models, upload test_data.csv, and compare the displayed scores to results/comparison.csv. Inspect class errors and download predictions. A missing target must trigger a readable message; a missing class must produce an explained N/A for AUC.

Publish through Streamlit Community Cloud

1. Sign in. Open streamlit.io/cloud and use your GitHub account.

2. Create app. Choose your real repository, branch (usually main), and app.py. The current documentation calls this Create app; the assignment calls it New App.

3. Match Python. Choose Python 3.12 under Advanced settings for this reference environment, or your own verified matching version.

4. Deploy. Read the logs if a dependency or artifact is missing. The repository must include every saved pipeline and the test CSV.

5. Verify evaluator access. Open the actual URL in a private/signed-out browser window and check that the interactive frontend, upload, dropdown, and metrics work.

Checks completed for this reference

Twenty Streamlit integration checks passed, covering all five metric panels, the comparison table, matrices, errors, valid upload data, absent-class AUC, and eight invalid-input cases. Pipeline.fit was disabled during these checks to detect unintended retraining.

Limit: CSV bytes were injected at the file-uploader boundary because AppTest lacks a native upload setter. Real browser file-picker transport, hosted deployment, and BITS execution still need your verification.

FINAL SUBMISSION

Assemble one PDF with real evidence

Complete the four parts below in exactly this order. This learning guide is not the submission PDF.

Order	Required content	Completion condition
1	GitHub repository link	Clickable; opens your actual source, dependencies, README, test CSV, and all models.
2	Live Streamlit app link	Clickable; opens your deployed interactive Community Cloud app.
3	ONE BITS Lab screenshot	Genuine screenshot showing your assignment execution within the lab.
4	Full README content	Final sections a-d, metric table, all observations, and overall winner.

Final verification

1. Open both links from the exported PDF, not just from your editor.

2. Check every model and upload test_data.csv in the hosted app.

3. Ensure the lab screenshot is readable and shows meaningful execution/results.

4. Include the complete final README, not only its URL or a screenshot.

5. Remove all pending placeholders and ensure reported scores match your own run.

6. Use the course submission action; do not leave the work as a draft.

Primary references

Dataset and attribution

Preprocessing and leakage

Multiclass ROC-AUC

Model persistence and versions

Streamlit deployment

Streamlit dependencies

Assignment requirements were read from the supplied ML_Assignment_2_AIML_26Oct26.pdf. Experimental scores come from this project's saved results, not from an external repository. The project includes the source archive, transformation code, split membership, and package versions for reproduction.